Abbozzo Simulatore

March 4, 2026

0.1 Assunzioni

- Viene modellizzato un sistema parts-to-picker: i pod mobili conteneti varie SKU ciascuno vengono portati a delle workstation fisse dai robot della fleet. Le SKU sono distribuite su vari pod.
- Non è considerato il numero di una data SKU su un pod (e quindi neanche dei vincoli di inventario o il task di replenishment), pertanto neanche il volume delle SKU ha importanza

1 Input Information

Sezione ispirata da [MXL18], articolo di presentazione del software RAWSim-O.

Il simulatore avrà bisogno di alcune informazione iniziali al partire dalle quali creare l'istanza di simulazione. La sezione è impostata seguendo quanto fatto da [MXL18] adattandolo al nostro caso semplificato.

1.1 Layout specification

Un primo input riguarda il layout fisico del sistema.
A questo livello ci sono vari parametri da fissare:

- P = numero di pod
- R = numero di robot
- W = numero di workstation
- [BBE17] introduce anche un parametro $\xi \in \{0.005, 0.05, 0.2\}$ che indica la frazione di SKU per pod. In questo senso ogni pod contiene un numero di SKU compreso in $[1, \lceil \xi P \rceil]$.

Una prima struttura dati necessaria è una matrice $N \times N$ ($P = N^2$) le cui entrate rapresentano il set di SKU contenute dal pod.

1.2 Scenario Specification

In questa fase vengono specificati dei parametri che permettono la generazione degli ordini.

Alcuni, tra cui [JLH23] e [JP23] e il simulatore RAWSim-O stesso, seguono la logica di generazione degli ordini presentata da [BBE17]. Per ogni ordine si genera inizialmente il numero di item da una distribuzione uniforme discreta in $[1, 0.1 * R + 1]$ (per lui R non è il numero di pod ma un valore inferiore che indica quanti pod sono necessari a soddisfare gli ordini, ed è fissato $R \in [5; 100]$) e poi vengono estratte le SKU. Ogni SKU è caratterizzata da un parametro di priorità (estratto da un esponenziale di parametro $\lambda = 0.5$) che determina la probabilità che venga estratta

$$p_i = \frac{\text{parametro di priorità SKU i}}{\text{somma dei parametri di priorità di tutte le SKU}}$$

.

1.3 Controller Configuration

Regola la parte di in che modo gestire l'ottimizzazione.

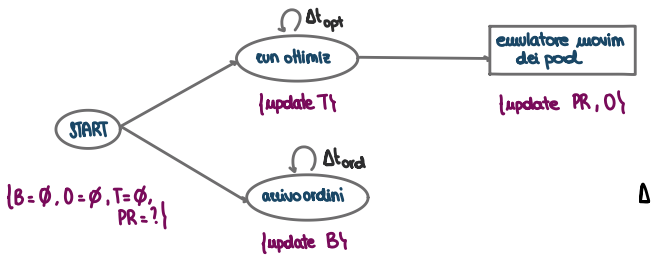
2 Struttura della Simulazione

Prima di entrare nei dettagli tecnici della simulazione, descrivo brevemente la dinamica del sistema.

Gli ordini arrivano in modo continuo (per semplicità assumeremo tempi di interarrivo costanti) e vanno ad inserirsi nel **backlog degli ordini**. A intervalli regolari viene eseguito un ottimizzatore che, considerando solo un sottoinsieme di ordini presenti nel backlog, determina l'assegnazione ordini-workstation-tempo e la sequenza con cui i pod devono visitare le diverse workstation.

La simulazione deve inoltre contenere un **emulatore del movimento dei pod**, ilcui compito è di prendere lo scheduling generato dall'ottimizzatore e simulare i movimenti dei robot incaricati di trasportare i pod verso le workstation assegnate.
Questo emulatore sarà descritto da uno stato contenete informazione sugli ordini in lavorazione nelle workstation, sui pod in movimentazione e sullo stato (busy o free) di un robot che sarà input dell'ottimizzatore quando verrà richiamato.

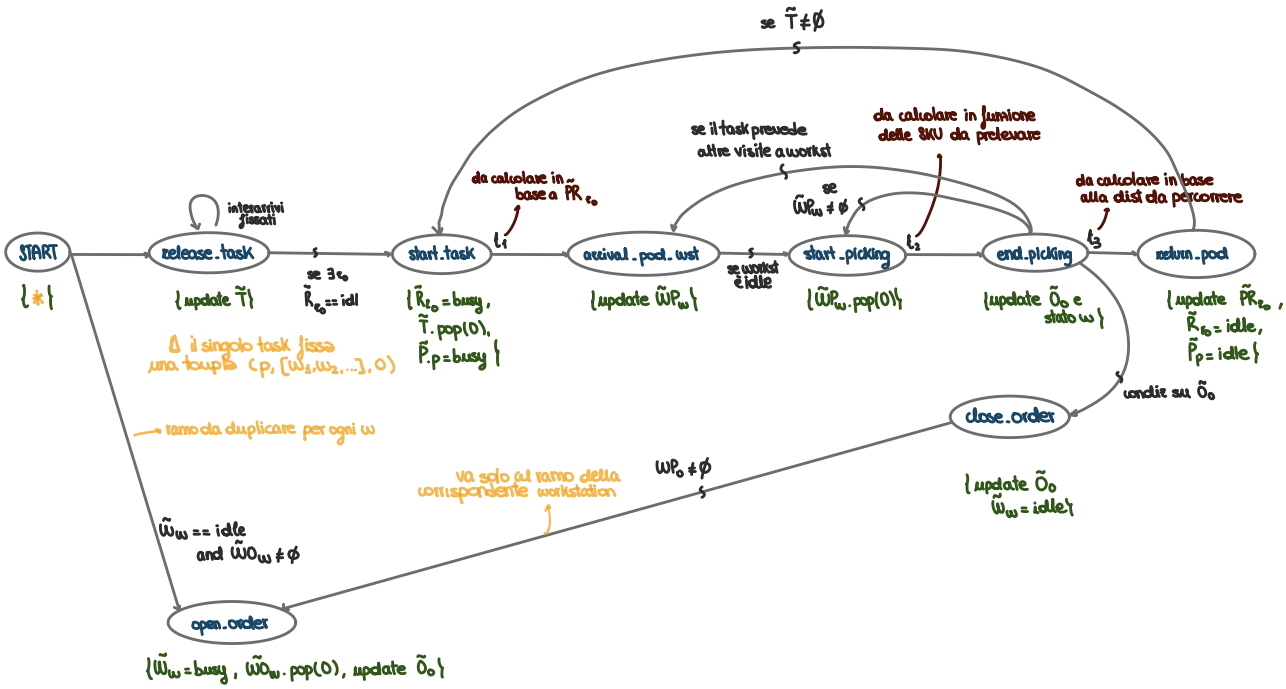
STATO: (B = backlog ordini , O = stato ordini già considerati nell'opt prec , PR = posizione robot , T = coda task)



Δ cun ottimiz sfrutta le info in B e O
emulatore movim dei pod sfrutta le info in T e PR e O

emulatore movim dei pod

STATO = ($\tilde{PR}$ = posizione robot , $\tilde{T}$ = coda dei task released , $\tilde{O}_o$ = info sul singolo ordine , $\tilde{P}_p$ = stato pod p , $\tilde{R}_r$ = stato robot r , $\tilde{W}_w$ = stato workstation w , $\tilde{WP}_w$ = coda dei pod alla workstation w , $\tilde{WO}_w$ = coda degli ordini alla workstation w)



* $\tilde{T}=\emptyset$, $\tilde{PR}=PR$, $\tilde{P}_p$ = idle $\forall p$, $\tilde{R}_r$ = idle $\forall r$, $\tilde{O}_o$ = { aperto se $o \in O$ / in coda se $o \notin O$ }, $\tilde{WP}_w = \emptyset$ $\forall w$, $\tilde{WO}_w$ da inizializzare in base ai risultati dell'opt

2.1 Emulatore del movimento dei robot

Questa è il modulo che a partire dallo scheduling ottenuto simula la parte di movimentazione dei pod. Immagino che l'ottimizzatore consideri variabili decisionali del tipo

$$\text{ordini-workstation-tempo} \quad x_{owt} = \begin{cases} 1 & \text{se allo step } t \text{ l'ordine } o \text{ è aperto alla workstation } w \\ 0 & \text{altrimenti} \end{cases}$$

$$\text{pod-workstation-tempo} \quad y_{iopwt} = \begin{cases} 1 & \text{se la SKU } i \text{ per l'ordine } o \text{ è già stata prelevata dal pod } p \text{ alla workstation } w \text{ allo ste} \\ 0 & \text{altrimenti} \end{cases}$$

(due delle variabili decisionali del modello matematico della Barnhart [BJS24]).

2.1.1 Come definisco lo stato?

Ad ogni istante devo tenere traccia di:

- **Stato degli Ordini**: per ogni ordine devo sapere il suo stato (in attesa, aperto, completato) e le SKU mancanti per il completamento
- **Stato dei Pod**: per ogni pod devo sapere il suo stato (assegnato o no)
- **Stato dei Robot**: per ogni robot devo sapere il suo stato (occupato o no) e la sua posizione se non occupato
- **Stato delle Workstation**: per ogni workstation devo sapere la coda degli ordini (determinata da x_{owt}), il suo stato (occupata o meno) e la coda dei pod in attesa di processamento (y_{pwt} mi determina la sequenza ideale ma essendo deterministici i travel time e i tempi di picking potrei avere dei ritardi)
- **Coda dei Task**: ogni task è definito dal pod che necessito, da un parametro di priorità (direi che dipende sia dalla distanza da percorrere che da quando mi serve in una workstation) e dalle destinazioni che devo raggiungere (penso ad una lista ordinata di workstation ad esempio). Questa coda ha come risorsa i robot (per semplicità un task viene allocato al robot libero più vicino).

2.1.2 Quali sono gli eventi?

- **ARRIVO_POD_WORKSTATION(p,w,r)**: evento che si crea quando un task viene allocato con timestamp $t = t_0 +$ tempo spostamento. Conseguenza: il pod viene incodato alla workstation
- **FINE_PICKING(p,w,r)**: evento che si crea quando un pod è incodato alla workstation con timestamp $t = t_0$+tempo picking. Conseguenza: o si genera un evento ARRIVO_POD_WORKSTATION(p,w,r) oppure uno RIENTRO_POD(p,r).
  Attenzione: alla fine di un picking ad una workstation ci deve essere un controllo se l'ordine lì assegnato può cambiare il suo status da*aperto* a *completato*.
- **RIENTRO_POD(p,r)**: evento che ha come conseguenza il cambio di stato del pod, il cambio di stato e posizione del robot.
  Attenzione: a seguito di questo evento bisogna guardare la possibilità di allocare altri task

2.1.3 Quali sono le code del sistema?

1. **Coda degli ordini alle workstation**: ogni workstation ha una sua coda degli ordini la cui priorità è determinata da x_{owt}.
   Ogni qualvolta inizia un servizio bisogna cabiare lo stato dell'ordine e della workstation e bisogna aggiungere elementi nella coda dei task.

2. **Coda dei pod alle workstation**: ogni workstation ha una sua coda deu pod la cui priorità è determinata da FIFO.
   Ogni qualvolta inizia un nuovo servizio bisogna creare un evento FINE_PICKING() e aggiornare lo stato dell'ordine presente alla workstation

3. **Coda dei task**: coda generale del sistema che viene servita dai robot.
   Ogni qualvolta inizia un nuovo servizio bisgna creare un evento ARRIVO_POD_WORKSTATION() e cambiare lo stato del pod e del robot.

3 Domande al prof

1. Io sto pensando alla simulazione come costruita a due livelli: il primo si occupa della generazione degli ordini e dell'ottimizzazione periodica, mentre il secondoe simula gli spostamenti dei pod. E questi due livelli devono in qualche modo interagire in quanto (1) lo stato attuale del sistema influenza alcuni vincoli dell'ottimizzatore (ad esempio se non ho finito di servire l'ordine 1 alla workstation 2 devo inserire un vincolo per cui $x_1, 2, 0 = 1$), e viceversa (2) l'output dell'ottimizzatore è l'input della simulazione dei movimenti.
   Per farli interagire senza incoerenze, posso assumere che quando parte lottimizzazione il sistema sia "congelato" e poi quando faccio ripartire la simulazione resetto la lista degli eventi?
   Alternativamente, potrei assumere (mi sembra più drastico) che l'ottimizzatore parta solo quando ho finito di servire tutti gli oridini considerati nell'ottimizzazione precedente.

2. Se considero le variabili y_{pwt} come valide indicazioni di scheduling dei pod, in quale istante della simulazione devo effettivamente attivare la richiesta di movimentazione del pod verso la workstation?
   Mi vengono in mente due approcci: (1) impostare una coda delle richieste che attinge ai robot come risorsa (problema: come decido chi ha la priorità tra pod necessari allo stesso step) oppure (2) generare richieste "anticipate" che tengano conto di quanto disti il pod dalla workstation.
   Seconda questione: Se ho dei robot liberi può avere senso che inizio a prelevare pod che mi serviranno?

4 Questioni a cui devo pensare

1. PROBLEMA: fare attenzione ai casi $y_{p,w_1,t} = 1$ e $y_{p,w_2,t+1} = 1 \longrightarrow$ forse risolto

2. PROBLEMA: se $y_{p,w_1,t} = 1$ e $y_{p,w_2,t+2} = 1$ ha effettivamente senso che il pod torni al suo posto? C'è qualcosa nell'ottimizzatore che si assicuri che ciò non accada?

References

[BJS24] Cynthia Barnhart, Alexandre Jacquillat, and Alexandria Schmid. "Robotic warehousing operations: a learn-then-optimize approach to large-scale neighborhood search". In: *ArXiv* abs/2408.16890 (2024). URL: https://api.semanticscholar.org/CorpusID:272310402.

[BBE17] Nils Boysen, Dirk Briskorn, and Simon Emde. "Parts-to-picker based order processing in a rack-moving mobile robots environment". In: *European Journal of Operational Research* 262.2 (2017), pp. 550–562. ISSN: 0377-2217. DOI: 10.1016/j.ejor.2017.03.053.

[JLH23] Guoshuai Jiao, Haobin Li, and Min Huang. "Online joint optimization of pick order assignment and pick pod selection in robotic mobile fulfillment systems". In: *Computers & Industrial Engineering* 175 (2023), p. 108856. ISSN: 0360-8352. DOI: 10.1016/j.cie.2022.108856.

[JP23] Jan-Erik Justkowiak and Erwin Pesch. "A column generation driven heuristic for order-scheduling and rack-sequencing in robotic mobile fulfillment systems". In: *Omega* 120 (2023), p. 102897. ISSN: 0305-0483. DOI: 10.1016/j.omega.2023.102897.

[MXL18] Marius Merschformann, Lin Xie, and Hanyi Li. "RAWSim-O: A Simulation Framework for Robotic Mobile Fulfillment Systems". In: *Logistics Research* 11.8 (2018). ISSN: 1865-035X. DOI: 10.23773/2018_8.